EN01

Satellite-Based Urban Tree-Equity Auditor & Planting Prioritization Engine

Product Development Plan

Version 1.0 | April 2026

Project Summary

A satellite-driven, multi-constraint optimization engine that routes tree-planting budgets to neighborhoods with the highest combined environmental and social impact, replacing intuition-based allocation with rigorous spatial analysis.

Timeline: 20 Weeks | Phases: 5 | Deliverables: 6 | Complexity: High

1. Executive Summary

Urban tree inequity is a measurable, addressable injustice. Lower-income neighborhoods consistently carry less tree canopy, resulting in higher surface temperatures, elevated cooling costs, poorer air quality, and worse health outcomes for residents who are already economically vulnerable. City planners and NGOs that want to intervene face a hard constraint: limited planting budgets that must be deployed where a single tree will achieve the greatest combined impact.

This product — the EN01 Satellite-Based Urban Tree-Equity Auditor & Planting Prioritization Engine — replaces intuition with evidence. It ingests public satellite imagery, computes vegetation and thermal indices at pixel level, fuses those layers with socioeconomic vulnerability data, and applies a budget-constrained optimization algorithm to generate a ranked, actionable intervention list that respects both tree count limits and water availability constraints.

Objective	Description
Equity Mapping	Quantify tree canopy gaps and thermal hotspots per neighborhood polygon with satellite-derived NDVI and LST.
Impact Scoring	Fuse environmental layers with socioeconomic vulnerability indices into a single weighted Tree Planting Impact Score.
Budget Optimization	Apply a Knapsack-style algorithm to select maximum-ROI planting zones within a hard tree count budget.
Constraint Filtering	Remove high-priority zones lacking municipal water access to ensure sapling survival feasibility.
Decision Dashboard	Deliver an interactive geospatial dashboard that maps results and exports prioritized intervention lists.

2. Problem Statement & Core Technical Complexity

2.1 The Problem

Tree inequity is both a symptom and a cause of urban inequality. The compounding effects are well-documented:

- Higher surface temperatures due to the urban heat island (UHI) effect in tree-deficient neighborhoods
- Elevated electricity costs for residents who rely on cooling in summer months
- Worse air quality as trees that would otherwise absorb vehicle and industrial pollutants are absent
- Poorer public health outcomes — respiratory illness, heat-related mortality — concentrated in economically vulnerable communities

City planners and NGOs operating with limited planting budgets currently make allocation decisions based on intuition, political geography, or administrative convenience rather than spatial evidence. The neighborhoods that most need intervention are the last to receive it.

2.2 Core Technical Complexity

This system operates at the intersection of four hard technical domains:

Domain	Technical Challenge
Remote Sensing	Pixel-level vegetation segmentation on multi-band public satellite imagery to compute NDVI and LST at neighborhood granularity, with cloud masking and seasonal compositing.
Deep Learning	Land cover classification distinguishing tree canopy, built surfaces, and bare ground from 10m-resolution imagery — balancing accuracy against inference cost.
Spatial Data Fusion	Joining raster-derived environmental metrics with polygon-level socioeconomic datasets of differing spatial resolutions and coordinate reference systems.
Combinatorial Optimization	Multi-constraint Knapsack selection across neighborhood polygons — NP-hard in general — requiring a heuristic or approximation approach at city scale, with a secondary water-availability feasibility filter.

3. Exact Deliverables

The project produces six core deliverables, each corresponding to a distinct technical layer:

D1 — Satellite Image Processing Pipeline

A reproducible pipeline that ingests Sentinel-2 Level-2A imagery from Google Earth Engine, applies cloud and shadow masking, computes NDVI from Band 8 (NIR) and Band 4 (Red), and retrieves Land Surface Temperature via the Landsat 8 TIRS thermal band with emissivity correction. Outputs are seasonal composite GeoTIFF rasters clipped to the city Area of Interest (AOI).

D2 — Pixel-Level Land Cover Segmentation Model

A semantic segmentation model (U-Net or DeepLab architecture) trained on a labelled dataset of Sentinel-2 image chips, distinguishing three classes: tree canopy, built/impervious surfaces, and bare ground. Model outputs a per-pixel classification raster at 10m resolution. Training uses publicly available land cover labels (e.g., ESA WorldCover, NLCD).

D3 — Data Fusion Layer & Tree Planting Impact Score

A spatial join and weighted scoring module that aggregates NDVI, LST, and canopy fraction per neighborhood polygon, then fuses these with at least one socioeconomic vulnerability layer (e.g., CDC Social Vulnerability Index, census income data). The composite Tree Planting Impact Score is computed as a configurable weighted sum. Score weights are exposed as user-adjustable parameters in the dashboard.

D4 — Budget Optimization Module

A Knapsack-style combinatorial optimizer that accepts the ranked neighborhood list, assigns a tree-count cost per zone (based on area and target canopy density), and selects the subset of zones maximizing total impact score within a user-specified tree count budget. Implements both a greedy approximation (for real-time UX) and an exact dynamic programming solution (for offline batch runs).

D5 — Water Constraint Filter

A spatial overlay that intersects prioritized planting zones with a municipal water infrastructure dataset (e.g., OpenStreetMap water mains, city utility GIS data). Zones outside a configurable buffer distance from water infrastructure are flagged as infeasible and removed from the final recommendation list, with an option to retain them in a secondary 'future infrastructure' tier.

D6 — Interactive Geospatial Dashboard

A web-based dashboard built on Streamlit or Dash with Folium/Leaflet map layers, displaying: (1) NDVI canopy desert heatmap, (2) LST thermal hotspot overlay, (3) socioeconomic vulnerability choropleth, (4) Tree Planting Impact Score per neighborhood polygon with color-graded ranking, (5) budget slider and optimization output, (6) water constraint toggle, and (7) exportable prioritized intervention list (CSV/GeoJSON).

4. System Architecture

4.1 Architecture Overview

The system is organized into five sequential layers with a shared geospatial data store. Each layer can be run independently in batch mode or triggered by the dashboard for on-demand computation.

Layer	Component	Responsibility
1 — Ingestion	GEE Image Pipeline	Query, cloud-mask, and export seasonal composites from Sentinel-2 and Landsat 8 for the target city AOI.
2 — Processing	NDVI / LST Calculator	Derive NDVI raster from NIR/Red bands; derive LST raster via TIRS thermal retrieval with split-window algorithm.
2 — Processing	Segmentation Model	Run trained U-Net over image chips to produce land cover classification raster; compute canopy fraction per polygon.
3 — Fusion	Spatial Join Engine	Aggregate raster metrics per neighborhood polygon; join with socioeconomic vector data; compute Impact Score.
4 — Optimization	Knapsack Optimizer	Score neighborhoods, apply water constraint filter, run greedy/DP Knapsack within user budget.
5 — Presentation	Geospatial Dashboard	Interactive Folium/Leaflet map with layer controls, budget slider, and intervention export.

4.2 Data Flow

- Sentinel-2 / Landsat 8 images ingested via GEE Python API, exported as Cloud-Optimized GeoTIFFs (COG) to Google Cloud Storage
- NDVI and LST rasters processed in GEE or locally via rasterio; outputs stored as COGs
- Segmentation model inference run on tiled image chips; tile predictions stitched into full-AOI raster
- Raster layers zonal-summarized to neighborhood polygons using rasterstats; joined with census/SVI data via GeoPandas
- Impact Score computed and stored in a GeoPackage (.gpkg) for fast dashboard loading
- Knapsack optimizer reads the scored GeoPackage; writes selected zones back as a filtered layer
- Dashboard loads GeoPackage layers, renders via Folium, serves via Streamlit

5. Development Roadmap

The project runs across 20 weeks in five sequential phases. Each phase concludes with a defined milestone and review gate.

Phase	Weeks	Deliverable / Goal	Exit Criteria
Phase 1	1–3	Project setup, GEE account provisioning, AOI selection (target city), Sentinel-2 image collection query, cloud masking validation, first NDVI composite export.	Clean NDVI raster for AOI with <5% cloud cover.
Phase 2	4–8	LST retrieval pipeline, land cover training data labeling (500+ chips), U-Net model training on GEE/local GPU, accuracy evaluation (target mIoU >0.80), model export.	Segmentation model with mIoU >0.80 on validation set.
Phase 3	9–12	Socioeconomic data acquisition (CDC SVI / census), zonal statistics aggregation, data fusion design, Tree Planting Impact Score formula definition and calibration, per-neighborhood scoring.	Scored GeoPackage for all city neighborhoods.
Phase 4	13–16	Knapsack optimizer implementation (greedy + DP), water infrastructure data acquisition, spatial buffer filter, multi-constraint optimization validation, unit and integration tests.	Optimizer produces correct ranked list under budget and water constraints.
Phase 5	17–20	Dashboard UI development (Streamlit + Folium), layer controls, budget slider, export functionality, user testing, performance optimization, documentation and final delivery.	Deployed dashboard accessible via browser; all 6 deliverables complete.

6. Data Model

6.1 Core Datasets

Dataset	Source	Format	Usage
Sentinel-2 Level-2A	ESA / GEE	Multi-band raster	NDVI computation, land cover segmentation input
Landsat 8 Collection 2	USGS / GEE	Multi-band raster	Land Surface Temperature via TIRS Band 10
ESA WorldCover / NLCD	ESA / USGS	GeoTIFF	Segmentation model training labels
CDC Social Vulnerability Index	CDC ATSDR	Shapefile / CSV	Socioeconomic vulnerability layer for Impact Score
Census Income Data	US Census / ACS	CSV + TIGER shp	Supplementary socioeconomic indicator
Municipal Water Infrastructure	OSM / City GIS	GeoJSON	Water constraint filter spatial overlay
Neighborhood Polygons	City Open Data	GeoJSON / SHP	Base spatial unit for all aggregation and scoring

6.2 Neighborhood Score Record

Each neighborhood polygon produces one scored record with the following fields:

Field	Type	Description
neighborhood_id	String	Unique identifier from city boundary dataset
geometry	Polygon	GeoJSON polygon in EPSG:4326
mean_ndvi	Float	Mean NDVI from seasonal composite; range -1 to +1
canopy_fraction	Float	Fraction of polygon classified as tree canopy by segmentation model (0-1)
mean_lst_celsius	Float	Mean Land Surface Temperature in Celsius
svi_score	Float	CDC Social Vulnerability Index score (0-1, higher = more vulnerable)
impact_score	Float	Composite weighted Tree Planting Impact Score (0-100)
trees_needed	Integer	Estimated tree count to reach target canopy fraction (Knapsack item weight)
water_access	Boolean	True if within 200m buffer of municipal water main

selected	Boolean	True if selected by Knapsack optimizer within current budget

7. Key Algorithms

7.1 NDVI Computation

NDVI = (NIR - Red) / (NIR + Red) computed band-by-band per pixel using Sentinel-2 Band 8 (NIR, 842nm) and Band 4 (Red, 665nm). Values range from -1.0 (water/built) to +1.0 (dense vegetation). Threshold: NDVI < 0.2 classified as non-vegetated. Seasonal compositing uses median pixel value across cloud-masked images within a 3-month window to reduce phenological and atmospheric noise.

7.2 Land Surface Temperature Retrieval

LST is derived from Landsat 8 TIRS Band 10 (10.6-11.19 μm) using the single-channel algorithm: LST = BT / (1 + (λ × BT / c2) × ln(ε)), where BT is brightness temperature, λ is band center wavelength, c2 is Planck's second radiation constant, and ε is land surface emissivity estimated from NDVI via the NDVI threshold emissivity method. Output in Kelvin, converted to Celsius.

7.3 Tree Planting Impact Score

The composite Impact Score is a configurable weighted sum of four normalized components:

Component	Default Weight	Direction	Rationale
NDVI deficit (1 - mean_ndvi)	0.30	Higher = better target	Neighborhoods with low NDVI benefit more from planting
LST excess (z-scored)	0.25	Higher = better target	Thermally stressed areas gain more from shade and evapotranspiration
SVI score	0.30	Higher = better target	Prioritizes vulnerable populations with least adaptive capacity
Canopy gap area (ha)	0.15	Higher = better target	Larger canopy gaps offer more planting opportunity

Score = 100 × (w1 × norm(NDVI_deficit) + w2 × norm(LST_excess) + w3 × SVI + w4 × norm(canopy_gap_area)). All components normalized to [0,1] before weighting. Weights configurable via dashboard.

7.4 Budget-Constrained Knapsack Optimization

Each neighborhood i is modeled as a Knapsack item with value v_i = impact_score_i and weight w_i = trees_needed_i. The budget B is the maximum total tree count. The objective is to select a subset S of neighborhoods maximizing sum(v_i for i in S) subject to sum(w_i for i in S) <= B and water_access_i = True for all i in S.

Two solver modes are implemented:

- Greedy approximation: Sort by impact_score / trees_needed descending; add items while budget permits. O(n log n). Used for real-time dashboard interaction.
- Dynamic programming exact solver: Standard 0/1 Knapsack DP table with dimensions n × B. Used for offline overnight batch runs. Feasible for B up to ~10,000 trees and n up to ~500 neighborhoods.
- Water constraint applied as a pre-filter before Knapsack: infeasible zones are removed from the item pool entirely before optimization runs.

8. Technology Stack

Layer	Technology	Justification
Satellite Data	Google Earth Engine (Python API)	Free access to full Sentinel-2 and Landsat archives; server-side NDVI/LST computation at scale without data download.
Segmentation	PyTorch + segmentation_models_pytorch	U-Net and DeepLab V3+ pre-trained backbones; easy fine-tuning on remote sensing chip datasets.
Raster Processing	rasterio, GDAL, NumPy	Standard geospatial raster I/O; efficient windowed reads for large COG files.
Vector / Spatial	GeoPandas, Shapely, rasterstats	Polygon operations, CRS transforms, and zonal statistics for raster-to-polygon aggregation.
Optimization	Python (custom DP + greedy)	0/1 Knapsack DP implemented in NumPy for performance; OR-Tools (Google) as optional drop-in for ILP formulation.
Dashboard	Streamlit + Folium + Plotly	Rapid Python-native dashboard development; Folium for Leaflet.js map layers; Plotly for charts.
Data Storage	GeoPackage (.gpkg), GCS	GeoPackage for local scored vector data; Google Cloud Storage for raster intermediates.
ML Training	Google Colab Pro / GEE	GPU-accelerated model training; GEE for large-scale image chip export.
Environment	Docker + conda	Reproducible environment for geospatial dependencies (GDAL, PROJ); containerized deployment.

9. Risk Register

Risk	Severity	Impact	Mitigation
GEE export quotas exceeded for large city AOI	HIGH	Delays in raster delivery block all downstream processing.	Tile exports spatially; schedule batch exports overnight; use COG streaming.
Segmentation model mIoU below 0.80 on target city imagery	HIGH	Canopy fraction estimates are unreliable, invalidating Impact Score.	Collect city-specific labeling samples; use transfer learning from ESA WorldCover checkpoint.
Socioeconomic data lacks neighborhood-level granularity	MEDIUM	Impact Score conflates dissimilar areas; equity targeting degrades.	Fall back to census tract level; interpolate via areal weighting.
Municipal water infrastructure GIS unavailable	MEDIUM	Water constraint filter cannot be applied; deliverable D5 incomplete.	Use OSM water mains as proxy; flag zones as 'unverified' rather than removing.
Knapsack DP intractable for large budget values	MEDIUM	Exact solver too slow for interactive use with B > 50,000 trees.	Default to greedy for real-time; cap DP at B=10,000 with user warning.
Cloud cover >30% over city during target season	LOW	Median composite NDVI underestimates vegetated areas.	Extend compositing window to 6 months; use Sentinel-1 SAR as cloud-free backup.
Dashboard performance slow with large vector datasets	LOW	Poor user experience; map rendering exceeds 10s.	Simplify polygon geometries; serve pre-rendered tile layers; lazy-load detail on click.

10. Team & Roles

Role	Phases	Responsibilities
Remote Sensing Engineer	1, 2	GEE pipeline, NDVI/LST computation, cloud masking, image chip export
ML / Deep Learning Engineer	2	Dataset labeling, U-Net training and validation, model export and deployment
Geospatial Data Analyst	3	Data acquisition, zonal statistics, socioeconomic fusion, Impact Score calibration
Software Engineer (Optimization)	4	Knapsack DP and greedy implementation, water constraint filter, unit tests
Full-Stack / Dashboard Engineer	5	Streamlit dashboard, Folium map layers, export functionality, UI testing
Project Manager / Domain Expert	1–5	Stakeholder liaison, AOI selection, weight calibration with city planners, documentation

11. Expected Outcome & Success Metrics

11.1 System Outcome

The EN01 system will deliver an operational planning tool that enables city agencies and NGOs to move from intuition-based to evidence-based tree planting allocation. The interactive dashboard will visualize canopy deserts, thermal hotspots, and social vulnerability layers on a unified map, compute a ranked Tree Planting Impact Score for every neighborhood, and output a prioritized intervention list that respects both budget and water feasibility constraints.

11.2 Quantitative Success Metrics

Metric	Target	Measurement Method
Segmentation model mean IoU (tree canopy class)	>= 0.80	Held-out test set evaluation
NDVI composite cloud-free coverage of AOI	>= 95%	QA band pixel count
Neighborhood scoring coverage	100% of city polygons	Count of null vs. scored records
Knapsack greedy response time (interactive)	< 2 seconds	Browser console timing
Dashboard map load time	< 5 seconds	Lighthouse / manual timing
Optimizer budget utilization efficiency (greedy vs. optimal gap)	< 5% suboptimal	Compare greedy vs. DP output on benchmark instances

End of Document